

WORKSHOP DE CIENCIA DE DATOS

INFORME DE PRÁCTICA

Detección de anomalías GPS con LSTM Autoencoder

Entrenamiento, definición de umbral y evaluación de un ataque GPS simulado

Estudiante	Alonzo Pinargote Roberth Alexander
Fecha de ejecución	30 de julio de 2026
Modalidad	Visual Studio Code - Jupyter Notebook
Materia	Inteligencia Artificial “A”
Docente	Oscar Gonzalez

Evidencias de ejecución, análisis y resultados

1. Introducción

En esta práctica se ejecutó un LSTM Autoencoder para detectar anomalías en secuencias de datos GPS. El modelo fue entrenado con 10 000 registros y cuatro variables derivadas de movimiento: velocidad, orientación, cambio de latitud y cambio de longitud. Posteriormente se definió un umbral basado en el error de reconstrucción y se simuló una alteración de la latitud para evaluar la capacidad de identificar un posible ataque.

2. Objetivos

2.1 Objetivo general

Implementar y evaluar un LSTM Autoencoder orientado a la detección de anomalías en datos GPS mediante el análisis del error de reconstrucción de secuencias temporales.

2.2 Objetivos específicos

- Preparar 10 000 registros del dataset unificado y construir secuencias de 30 observaciones.
- Entrenar un autoencoder recurrente para reconstruir patrones considerados normales.
- Calcular un umbral de anomalía mediante el percentil 99.5 del error MAE.
- Simular una modificación de la latitud dentro del conjunto de prueba.
- Evaluar el modelo con precision, recall, F1-score y matriz de confusión.

3. Configuración empleada

Parámetro	Valor utilizado
Entorno	Visual Studio Code con Jupyter
Registros	10 000
Variables	speed, course, delta_lat, delta_lon
Ventana temporal	30 registros
División	80 % entrenamiento / 20 % prueba
Arquitectura	LSTM Autoencoder de 13 188 parámetros
Entrenamiento	3 épocas; batch size 64
Umbral	Percentil 99.5 del MAE
Ataque simulado	Incremento de 0.03 en latitud desde la mitad de prueba

4. Desarrollo de la práctica

4.1 Arquitectura del modelo

La arquitectura está formada por una LSTM codificadora de 32 unidades, una capa Dropout, un RepeatVector que reconstruye la longitud de 30 pasos, una segunda LSTM decodificadora y una capa TimeDistributed que genera nuevamente las cuatro variables de entrada. El modelo contiene 13 188 parámetros entrenables.

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 32)	4,736
dropout (Dropout)	(None, 32)	0
repeat_vector (RepeatVector)	(None, 30, 32)	0
lstm_1 (LSTM)	(None, 30, 32)	8,320
dropout_1 (Dropout)	(None, 30, 32)	0
time_distributed (TimeDistributed)	(None, 30, 4)	132

Total params: 13,188 (51.52 KB)

Trainable params: 13,188 (51.52 KB)

Non-trainable params: 0 (0.00 B)

Figura 1. Resumen de la arquitectura secuencial del LSTM Autoencoder.

4.2 Entrenamiento del modelo

El entrenamiento se realizó durante tres épocas, sin mezclar las secuencias, con un 10 % reservado para validación. La pérdida descendió en cada época y EarlyStopping restauró los pesos correspondientes a la tercera época, que obtuvo el mejor valor de validación.

```

2do expositor > LSTM_Autoencoder_Deteccion_Anomalias_VSCode.ipynb > M Detección de anomalías GPS con LSTM Autoe
Generate + Code + Markdown | Run All Restart Clear All Outputs ... 3.1
10. Entrenar el modelo

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=2,
    restore_best_weights=True,
    verbose=1,
)

print("🚀 Iniciando entrenamiento de 3 épocas...")

history = model.fit(
    X_train_seq,
    X_train_seq,
    epochs=3,
    batch_size=64,
    validation_split=0.1,
    callbacks=[early_stop],
    shuffle=False,
    verbose=1,
)

[11] ✓ 17.6s

... 🚀 Iniciando entrenamiento de 3 épocas...
Epoch 1/3
113/113 ————— 11s 40ms/step - loss: 0.0863 - val_loss: 0.0506
Epoch 2/3
113/113 ————— 3s 30ms/step - loss: 0.0336 - val_loss: 0.0424
Epoch 3/3
113/113 ————— 3s 30ms/step - loss: 0.0258 - val_loss: 0.0309
Restoring model weights from the end of the best epoch: 3.

```

Figura 2. Ejecución del entrenamiento durante tres épocas.

4.3 Convergencia del entrenamiento

11. Gráfica de entrenamiento

```
plt.figure(figsize=(10, 5))

plt.plot(
    history.history["loss"],
    label="Pérdida de entrenamiento",
)

plt.plot(
    history.history["val_loss"],
    label="Pérdida de validación",
)

plt.title(
    "Convergencia del modelo LSTM Autoencoder"
)

plt.xlabel("Época")
plt.ylabel("Error MSE")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Figura 3. Código empleado para graficar las pérdidas.

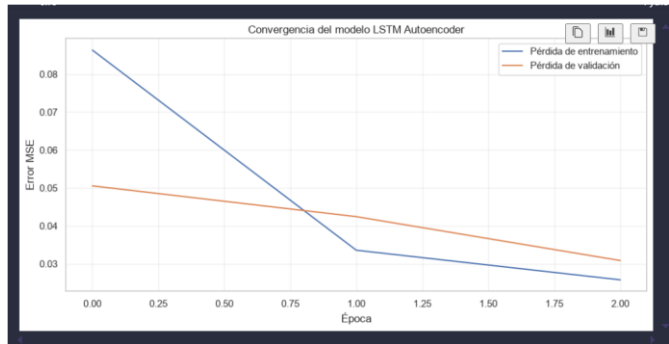


Figura 4. Curvas de entrenamiento y validación.

La pérdida de entrenamiento pasó de 0.0863 a 0.0258, mientras que la pérdida de validación descendió de 0.0506 a 0.0309. Las dos curvas disminuyen sin mostrar una separación creciente, por lo que no se observa sobreajuste evidente en estas tres épocas. Sin embargo, una buena reconstrucción no garantiza por sí sola una buena detección de ataques.

4.4 Definición del umbral de anomalía

13. Gráfica del umbral

```
plt.figure(figsize=(10, 5))

sns.histplot(
    train_mae,
    bins=40,
    kde=True,
)

plt.axvline(
    UMBRAL,
    linestyle="--",
    linewidth=2,
    label=f"Umbral = {UMBRAL:.4f}",
)

plt.title(
    "Distribución del error de reconstrucción"
)

plt.xlabel("Error MAE")
plt.ylabel("Frecuencia")
plt.legend()
plt.tight_layout()
plt.show()
```

Figura 5. Código para representar la distribución del error.

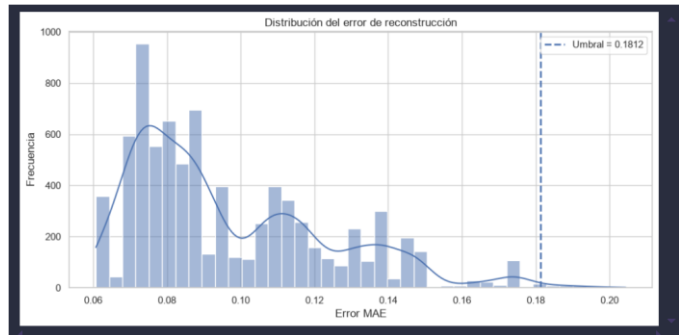


Figura 6. Distribución del MAE y umbral calculado.

El umbral obtenido fue 0.1812. La mayoría de los errores normales se concentró entre 0.06 y 0.15, mientras que solo una pequeña fracción superó el límite. El uso del percentil 99.5 reduce las falsas alarmas, pero también puede hacer que anomalías moderadas queden sin detectar.

5. Resultados obtenidos

Indicador	Resultado
Parámetros entrenables	13 188
Épocas ejecutadas	3
Loss final de entrenamiento	0.0258
Loss final de validación	0.0309
Umbral MAE	0.1812
Accuracy	0.51
Recall de ataques	0.10
F1-score de ataques	0.17
Ataques detectados	98 de 1 000
Ataques no detectados	902 de 1 000

El resultado global fue limitado. El modelo clasificó correctamente la mayoría de las secuencias normales, pero solo identificó 98 de los 1 000 ataques simulados. Esto explica el recall de 0.10 para la clase Ataque y el F1-score de 0.17, aunque la precisión de las alertas emitidas alcanzó 0.61.

5.1 Reporte de clasificación

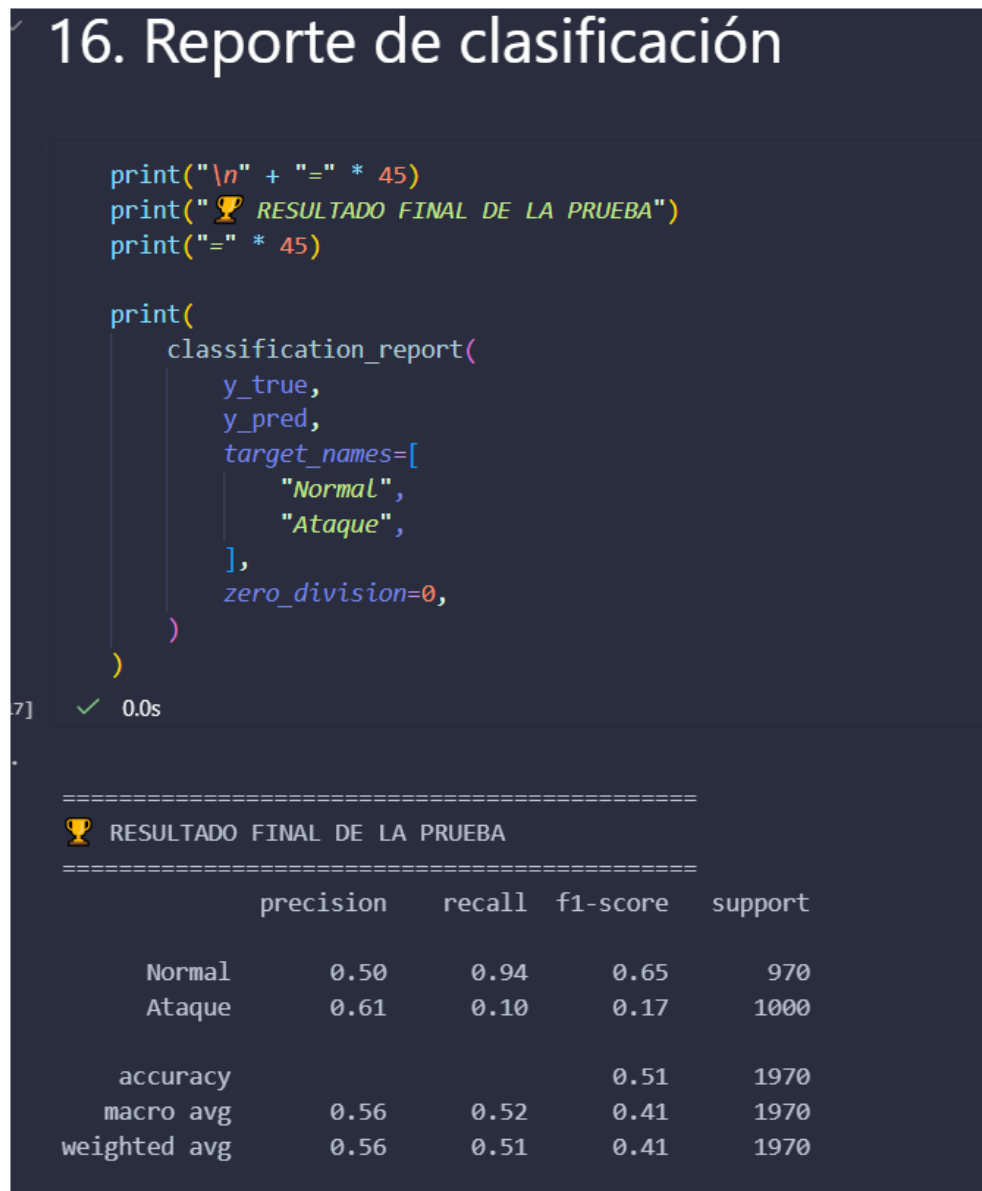


Figura 7. Reporte de clasificación obtenido en la prueba.

El reporte confirma un comportamiento desequilibrado: la clase Normal alcanzó un recall de 0.94, mientras la clase Ataque obtuvo solo 0.10. La accuracy de 0.51 está muy cerca de una decisión aleatoria para un conjunto casi balanceado, por lo que no debe interpretarse como un resultado satisfactorio.

5.2 Matriz de confusión

```

17. Matriz de confusión

matriz = confusion_matrix(
    y_true,
    y_pred,
    labels=[0, 1],
)

plt.figure(figsize=(6, 5))

sns.heatmap(
    matriz,
    annot=True,
    fmt="d",
    cbar=False,
    xticklabels=[
        "Normal",
        "Ataque",
    ],
    yticklabels=[
        "Normal",
        "Ataque",
    ],
)

plt.title(
    "Matriz de confusión final"
)

plt.xlabel("Predicción")
plt.ylabel("Valor real")
plt.tight_layout()
plt.show()

```

Figura 8. Código de generación de la matriz de confusión.

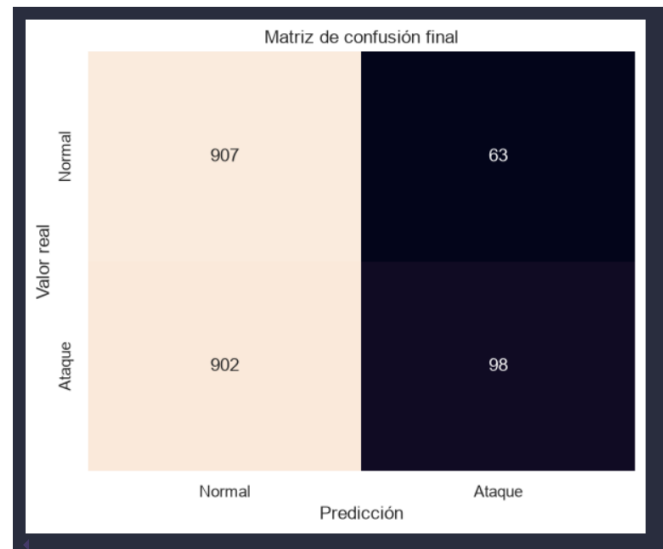


Figura 9. Matriz de confusión final.

La matriz registró 907 normales correctamente identificados, 63 falsas alarmas, 98 ataques detectados y 902 ataques clasificados como normales. En términos prácticos, el modelo dejó pasar el 90.2 % de los ataques simulados. Por ello, no debería considerarse adecuado para un sistema de seguridad sin modificar la simulación, las variables y el criterio de decisión.

5.3 Representación temporal de las anomalías

La última visualización compara el error de reconstrucción de cada secuencia con el umbral y sombrea la zona donde se aplicó el ataque. Las siguientes capturas muestran la construcción del gráfico final.

18. Gráfica final de anomalías

```
plt.figure(figsize=(12, 6))

plt.plot(
    test_mae,
    label="Error de reconstrucción",
)

plt.axhline(
    UMBRAL,
    linestyle="--",
    linewidth=2,
    label=f"Umbral = {UMBRAL:.4f}",
)

# Ajustar la posición del ataque al tamaño de las secuencias
inicio_ataque_grafica = max(
    0,
    indice_ataque - TIME_STEPS,
)

plt.axvspan(
    inicio_ataque_grafica,
    len(test_mae),
    alpha=0.2,
    label="Zona del ataque simulado",
)

plt.title(
    "Detección final de anomalías"
)
```

Figura 10. Primera parte del código de la gráfica final.

```
plt.xlabel("Secuencia temporal")
plt.ylabel("Error de reconstrucción")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(
    "\n✅ Prueba ejecutada correctamente "
    "con 10 000 registros y 3 épocas."
)
```

Figura 11. Segunda parte y confirmación de ejecución.

La lógica conserva la posición ajustada del ataque respecto a la ventana temporal de 30 registros y permite observar qué secuencias superan el umbral.

5.4 Análisis de la detección final

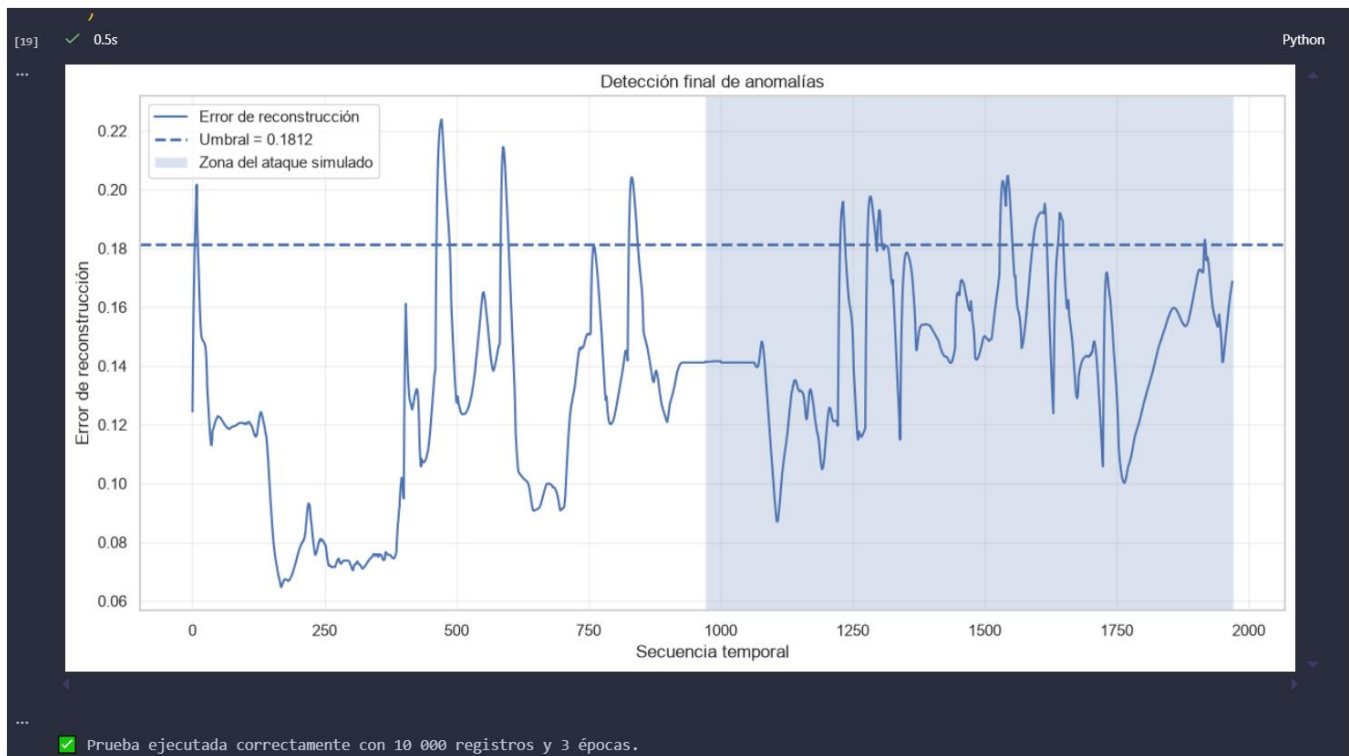


Figura 12. Error de reconstrucción, umbral y zona del ataque simulado.

Dentro de la zona sombreada existen varios picos por encima de 0.1812, pero la mayor parte de la señal permanece debajo del umbral. El desplazamiento constante de la latitud produce un cambio fuerte al inicio; después, al trabajar con `delta_lat`, las diferencias vuelven a parecer similares a las normales. Esta característica ayuda a explicar por qué el recall del ataque fue tan bajo.

6. Concepto clave aprendido durante el Workshop

Aprendí que en los modelos de detección de anomalías no basta con lograr una pérdida baja durante el entrenamiento. La simulación del ataque, las variables utilizadas y el umbral elegido determinan si el sistema realmente puede reconocer el comportamiento anómalo.

7. Tema más desafiante

Lo más desafiante fue interpretar la relación entre las secuencias temporales, el error de reconstrucción y el umbral, especialmente al comprobar que una buena convergencia del modelo no se tradujo en una detección efectiva de los ataques.

8. Limitaciones y mejoras propuestas

- El desplazamiento permanente de latitud se detecta principalmente en el instante inicial cuando se usan diferencias entre coordenadas.
- El umbral del percentil 99.5 favorece pocas falsas alarmas, pero reduce la sensibilidad frente a anomalías menos extremas.
- Las primeras 10 000 filas pueden pertenecer a una sola fuente del dataset y no representar todos los escenarios de movilidad.
- No se dispone de identificadores de vehículo, viaje o tiempo que garanticen continuidad real entre las secuencias.
- Se recomienda simular ataques graduales, intermitentes y sobre varias variables, además de comparar distintos umbrales.

9. Conclusiones

- El LSTM Autoencoder convergió correctamente y redujo la pérdida de validación hasta 0.0309.
- El umbral de 0.1812 permitió reconocer con alta frecuencia los registros normales, con 907 aciertos sobre 970.
- La detección de ataques fue insuficiente: se identificaron 98 y se omitieron 902.
- La accuracy de 0.51 confirma que la convergencia del entrenamiento no garantiza utilidad operativa.
- El modelo constituye una prueba inicial válida, pero necesita una simulación más realista, variables adicionales y ajuste del umbral antes de considerarse un detector confiable.